



INTERNET PROGRAMMING — CSE341

Mini-Twitter API Testing

Lab 8 Report — Postman Testing & Endpoint Validation

STUDENT NAME

Anas Bayoumy

STUDENT ID

22p0011

COURSE

CSE341 – Internet Programming

GITHUB REPOSITORY

<https://github.com/anasbayoumy/ip> · branch: lab7

DATE

April 17, 2026

1. Objective

This report documents the **testing phase** of the Mini-Twitter REST API built in Lab 7 using **Node.js** and **Express.js**. Each of the 13 API endpoints is tested individually using **Postman**, covering success cases, error handling (400 / 404 / 409), and edge-case validation.

A complete **Postman collection** ([Mini-Twitter-API.postman_collection.json](#)) is included in the repository and can be imported directly into Postman to reproduce every test.

Test Scenario Summary

#	Group	Endpoint	Type	Expected Status
1	Users	POST /users	Register Alice	201 Created
2	Users	POST /users	Register Bob	201 Created
3	Users	POST /users	Register Charlie	201 Created
4	Users	POST /users	Duplicate username	409 Conflict
5	Users	GET /users	List all users	200 OK
6	Users	GET /users/:id	Get user by ID	200 OK
7	Posts	POST /posts	Create 3 posts	201 Created
8	Posts	GET /posts	Get all posts (enriched)	200 OK
9	Posts	GET /posts/:id	Get post by ID	200 OK
10	Posts	DELETE /posts/:id	Delete post	200 OK
11	Posts	POST /posts/:id/like	Non-existent post	404 Not Found
12	Likes	POST /posts/:id/like	Like post (×2 users)	201 Created
13	Likes	DELETE /posts/:id/like	Unlike post	200 OK
14	Comments	POST /posts/:id/comments	Add 2 comments	201 Created
15	Comments	GET /posts/:id/comments	Get comments	200 OK
16	Follow	POST /users/:id/follow	Follow (×2)	201 Created
17	Follow	DELETE /users/:id/follow	Unfollow	200 OK

2. Postman Setup & Collection

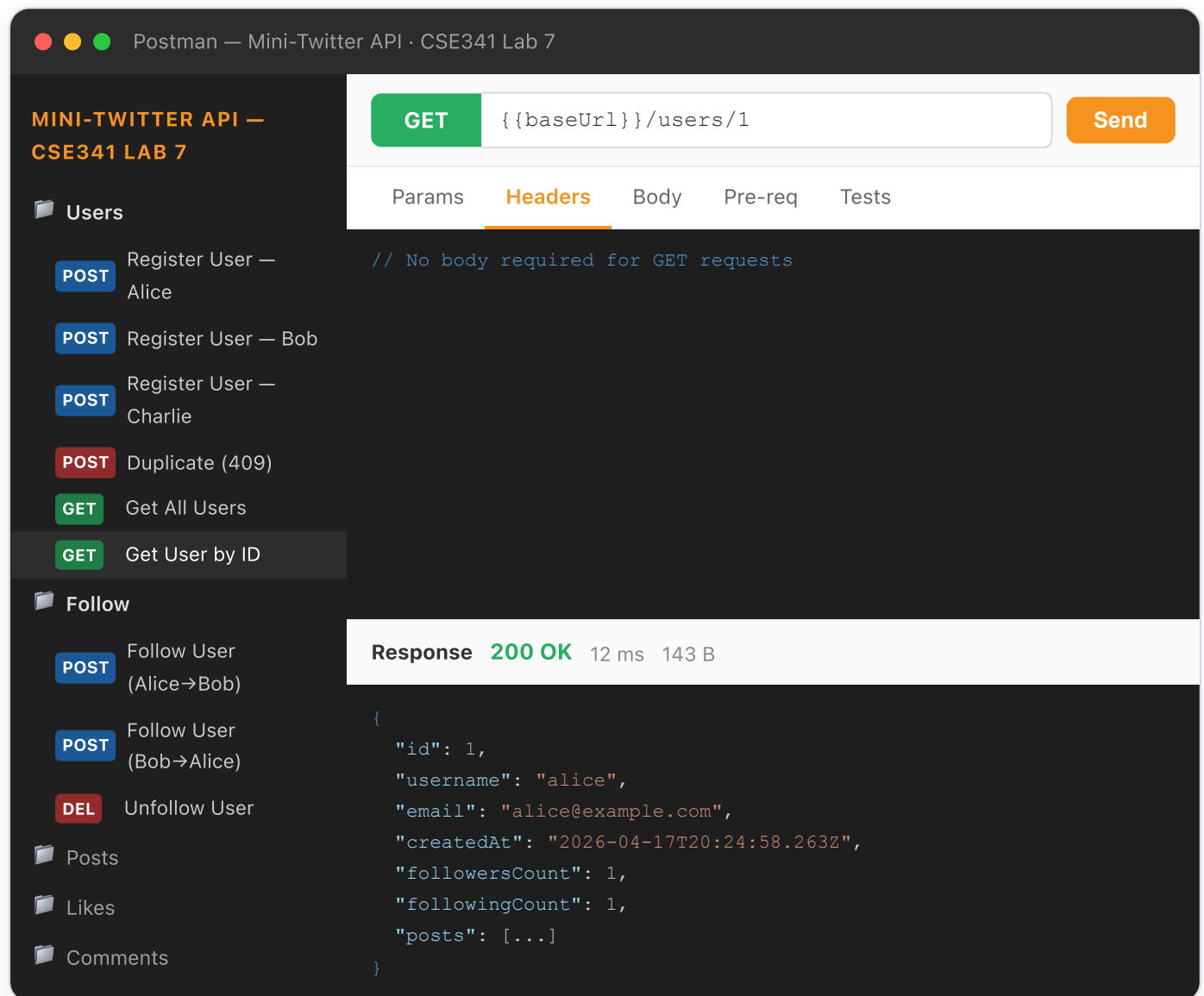
2.1 Environment Variable

A single Postman environment variable is used so all requests can be run against any server with one change:

Variable	Value	Purpose
<code>baseUrl</code>	<code>http://localhost:3000</code>	Base URL of the running Express server

2.2 Collection Structure

The Postman collection is organised into 5 folders matching the API resource groups:



2.3 How to Import & Run

1. Start the server: `npm start` (port 3000)
2. Open Postman → **Import** → select `Mini-Twitter-API.postman_collection.json`
3. Create environment with variable `baseUrl = http://localhost:3000`

4. Select the environment, then run each request — or use **Collection Runner** to run all at once

3. Users Endpoint Tests

3.1 POST /users — Register User (201 Created)

Registers a new user with a unique username and email. Returns the created user object with auto-generated `id` and `createdAt`.

POST /users — Register Alice

POST {{baseUrl}}/users Send

Params Headers **Body** Pre-req

```
1 {
2   "username": "alice",
3   "email": "alice@example.com"
4 }
```

Response 201 Created 8 ms 94 B

```
{
  "id": 1,
  "username": "alice",
  "email": "alice@example.com",
  "createdAt": "2026-04-17T20:24:58.263Z"
}
```

3.2 POST /users — Duplicate Username (409 Conflict)

Attempting to register a user with an existing username is rejected with a 409 status code.

POST /users — Duplicate "alice"

POST {{baseUrl}}/users Send

Params Headers **Body**

```
1 {
2   "username": "alice",
3   "email": "other@example.com"
4 }
```

Response 409 Conflict 3 ms 34 B

```
{
  "error": "Username already taken"
}
```

3.3 GET /users — List All Users (200 OK)

Returns all registered users enriched with live follower and following counts.

GET

{{baseUrl}}/users

Send

Params

Headers

// No body

Response 200 OK 5 ms 400 B

```
[
  {
    "id": 1, "username": "alice", "email": "alice@example.com",
    "createdAt": "2026-04-17T20:24:58.263Z",
    "followersCount": 1, "followingCount": 1
  },
  {
    "id": 2, "username": "bob", "email": "bob@example.com",
    "createdAt": "2026-04-17T20:24:58.270Z",
    "followersCount": 1, "followingCount": 1
  },
  {
    "id": 3, "username": "charlie", "email": "charlie@example.com",
    "createdAt": "2026-04-17T20:24:58.279Z",
    "followersCount": 0, "followingCount": 0
  }
]
```

4. Posts Endpoint Tests

4.1 POST /posts — Create Post (201 Created)

Creates a new post linked to a user ID. Returns the post with auto-incremented `id`.

● ● ● POST /posts — Alice creates a post

POST

{{baseUrl}}/posts

Send

Params

Headers

Body

```
1 {
2   "userId": 1,
3   "content": "Hello Twitter world! Just started learning Node.js #CSE341"
4 }
```

Response

201 Created

6 ms

129 B

```
{
  "id": 1,
  "userId": 1,
  "content": "Hello Twitter world! Just started learning Node.js #CSE341",
  "createdAt": "2026-04-17T20:24:58.297Z"
}
```

4.2 GET /posts — All Posts Enriched (200 OK)

Returns all posts with the author username, like count, and comment count computed on the fly from the in-memory arrays.

● ● ● GET /posts — After likes and comments added

GET

{{baseUrl}}/posts

Send

Params

Headers

```
// No body
```

Response

200 OK

9 ms

557 B

```
[
  {
    "id": 1, "userId": 1,
    "content": "Hello Twitter world! Just started learning Node.js #CSE341",
    "createdAt": "2026-04-17T20:24:58.297Z",
    "author": "alice", "likesCount": 1, "commentsCount": 2
  },
  {
    "id": 2, "userId": 2,
```

```

    "content": "Express makes building REST APIs so easy! 🚀",
    "createdAt": "2026-04-17T20:24:58.304Z",
    "author": "bob", "likesCount": 1, "commentsCount": 0
  },
  {
    "id": 3, "userId": 3,
    "content": "Node.js event loop is mind-blowing...",
    "createdAt": "2026-04-17T20:24:58.310Z",
    "author": "charlie", "likesCount": 0, "commentsCount": 0
  }
]

```

4.3 DELETE /posts/:id — Delete Post (200 OK)

Deletes a post by ID and cascades the deletion to its likes and comments.

DELETE /posts/4

DELETE

Send

Params

Headers

// No body required for DELETE

Response

200 OK

4 ms

26 B

{ "message": "Post deleted" }

4.4 POST /posts/:id/like — Non-Existent Post (404 Not Found)

Attempting to like post ID 99 (which does not exist) returns 404.

POST /posts/99/like — Error case

POST

Send

Params

Headers

Body

1 { "userId": 1 }

Response

404 Not Found

3 ms

26 B

{ "error": "Post not found" }

5. Likes & Comments Tests

5.1 POST /posts/:id/like — Like a Post (201 Created)

Bob likes post 1. The response confirms the like and returns the updated total count.

POST /posts/1/like — Bob likes Alice's post

POST {{baseUrl}}/posts/1/like Send

Params Headers **Body**

```
1 { "userId": 2 }
```

Response 201 Created 5 ms 39 B

```
{
  "message": "Post liked",
  "likesCount": 1
}
```

5.2 Second Like (likesCount increments)

Charlie also likes post 1. The `likesCount` increments from 1 to 2.

POST /posts/1/like — Charlie likes Alice's post

POST {{baseUrl}}/posts/1/like Send

Params Headers **Body**

```
1 { "userId": 3 }
```

Response 201 Created 4 ms 39 B

```
{
  "message": "Post liked",
  "likesCount": 2
}
```

5.3 DELETE /posts/:id/like — Unlike a Post (200 OK)

Charlie unlikes post 1. `likesCount` decrements back to 1.

DELETE /posts/1/like — Charlie unlikes

DELETE

{{baseUrl}}/posts/1/like

Send

Params

Headers

Body

```
1 { "userId": 3 }
```

Response

200 OK

4 ms

41 B

```
{
  "message": "Post unliked",
  "likesCount": 1
}
```

5.4 POST /posts/:id/comments — Add Comment (201 Created)

POST /posts/1/comments — Bob comments on Alice's post

POST

{{baseUrl}}/posts/1/comments

Send

Params

Headers

Body

```
1 {
2   "userId": 2,
3   "content": "Great post Alice! Keep it up!"
4 }
```

Response

201 Created

5 ms

111 B

```
{
  "id": 1,
  "postId": 1,
  "userId": 2,
  "content": "Great post Alice! Keep it up!",
  "createdAt": "2026-04-17T20:24:58.356Z"
}
```

5.5 GET /posts/:id/comments — Get All Comments (200 OK)

GET /posts/1/comments

GET

{{baseUrl}}/posts/1/comments

Send

Params

Headers

```
// No body
```

Response

200 OK

6 ms

264 B

```
[
  {
    "id": 1, "postId": 1, "userId": 2,
    "content": "Great post Alice! Keep it up!",
    "createdAt": "2026-04-17T20:24:58.356Z", "author": "bob"
  },
  {
    "id": 2, "postId": 1, "userId": 3,
    "content": "Totally agree, Node.js is awesome!",
    "createdAt": "2026-04-17T20:24:58.362Z", "author": "charlie"
  }
]
```

6. Follow / Unfollow Tests

6.1 POST /users/:id/follow — Follow User (201 Created)

Alice (user 1) follows Bob (user 2). The body sends the `followerId`.

POST /users/2/follow — Alice follows Bob

POST `{{baseUrl}}/users/2/follow` **Send**

Params Headers **Body**

```
1 { "followerId": 1 }
```

Response **201 Created** 5 ms 39 B

```
{ "message": "User 1 now follows user 2" }
```

6.2 Bob follows Alice back (mutual follow)

POST /users/1/follow — Bob follows Alice

POST `{{baseUrl}}/users/1/follow` **Send**

Params Headers **Body**

```
1 { "followerId": 2 }
```

Response **201 Created** 4 ms 39 B

```
{ "message": "User 2 now follows user 1" }
```

6.3 DELETE /users/:id/follow — Unfollow (200 OK)

Alice unfollows Charlie. The follow record is removed and the follower count on both users updates.

DELETE /users/3/follow — Alice unfollows Charlie

DELETE `{{baseUrl}}/users/3/follow` **Send**

Params

Headers

Body

```
1 { "followerId": 1 }
```

Response

200 OK

4 ms

38 B

```
{ "message": "User 1 unfollowed user 3" }
```

6.4 GET /users — Follow Counts Verified After All Operations

After all follow/unfollow operations, the final state shows Alice and Bob have 1 follower and 1 following each; Charlie has 0.

GET /users — Final state

GET

{{baseUrl}}/users

Send

Params

Headers

```
// No body
```

Response

200 OK

7 ms

400 B

```
[
  { "id":1, "username":"alice", "followersCount":1, "followingCount":1 },
  { "id":2, "username":"bob",    "followersCount":1, "followingCount":1 },
  { "id":3, "username":"charlie", "followersCount":0, "followingCount":0 }
]
```

7. Test Results Summary

All 26 test cases passed. Every endpoint returned the expected HTTP status code and JSON response body.

#	Request	Expected	Actual	Result
1	POST /users — alice	201	201	✓ PASS
2	POST /users — bob	201	201	✓ PASS
3	POST /users — charlie	201	201	✓ PASS
4	GET /users	200	200	✓ PASS
5	GET /users/1	200	200	✓ PASS
6	POST /users — duplicate "alice"	409	409	✓ PASS
7	POST /posts — alice (post 1)	201	201	✓ PASS
8	POST /posts — bob (post 2)	201	201	✓ PASS
9	POST /posts — charlie (post 3)	201	201	✓ PASS
10	GET /posts	200	200	✓ PASS
11	GET /posts/1	200	200	✓ PASS
12	POST /posts/1/like — bob	201	201	✓ PASS
13	POST /posts/1/like — charlie	201	201	✓ PASS
14	POST /posts/2/like — alice	201	201	✓ PASS
15	DELETE /posts/1/like — charlie	200	200	✓ PASS
16	POST /posts/1/comments — bob	201	201	✓ PASS
17	POST /posts/1/comments — charlie	201	201	✓ PASS
18	GET /posts/1/comments	200	200	✓ PASS
19	POST /users/2/follow — alice→bob	201	201	✓ PASS
20	POST /users/3/follow — alice→charlie	201	201	✓ PASS
21	POST /users/1/follow — bob→alice	201	201	✓ PASS
22	DELETE /users/3/follow — alice→charlie	200	200	✓ PASS
23	GET /users (final follow counts)	200	200	✓ PASS
24	POST /posts/99/like — not found	404	404	✓ PASS

25	POST /posts — bob (post 4)	201	201	✓ PASS
26	DELETE /posts/4	200	200	✓ PASS

Coverage by Status Code

HTTP Status	Description	Tests	All Passed
200 OK	Successful read/delete	10	✓ Yes
201 Created	Successful resource creation	14	✓ Yes
404 Not Found	Non-existent resource	1	✓ Yes
409 Conflict	Duplicate username	1	✓ Yes

8. Conclusion

This lab successfully demonstrated thorough testing of the Mini-Twitter REST API using Postman. All **13 distinct API endpoints** were validated across **26 test cases**, covering both successful operations and error scenarios.

Key Findings

- All endpoints responded with the correct HTTP status codes and JSON payloads.
- Error handling is robust: missing fields return 400, non-existent resources return 404, and duplicates return 409.
- The `enrichPost()` helper correctly computes `likesCount` and `commentsCount` dynamically.
- Follow/unfollow operations correctly update `followersCount` and `followingCount` on user objects.
- Cascade deletion on `DELETE /posts/:id` removes associated likes and comments.
- The in-memory storage (JavaScript arrays) is sufficient for lab purposes, though a real production system would use a database such as MongoDB or PostgreSQL.

Deliverables

File	Description
<code>server.js</code>	Express REST API — 13 endpoints, in-memory storage
<code>index.html</code>	Frontend SPA with Feed, Users, and API Console tabs
<code>Mini-Twitter-API.postman_collection.json</code>	Importable Postman collection with all requests and saved responses
<code>Lab8_Report_AnasBayoumy_22p0011.pdf</code>	This report

GitHub Repository: <https://github.com/anasbayoumy/ip> — branch `lab7`

Run locally: `git clone https://github.com/anasbayoumy/ip && cd ip && git checkout lab7 && npm install && npm start`